

Fundamentos de modelos digitales de elevación y elaboración cartográfica

Wilder Clavijo

Maria José Duarte

Suli Vanessa Gonzales

Damián Nicolas Piracún

Table of contents

1 Algoritmo de interpolacion IDW:	1
2 Algoritmo de interpolacion con correccion SAGA:	2

1 Algoritmo de interpolacion IDW:

Para este ejercicio de reproducibilidad se tomó como base el algoritmo IDW, sin embargo (y como se mostró en el ejemplo), sus limitaciones son evidentes en función de la correlación de los datos (Figura 1), en donde se evidencia que el error de predicción es alto y la correlación entre puntos es casi imperceptible, lo que se traduce en un modelo de elevación con una representación visual poco realista.

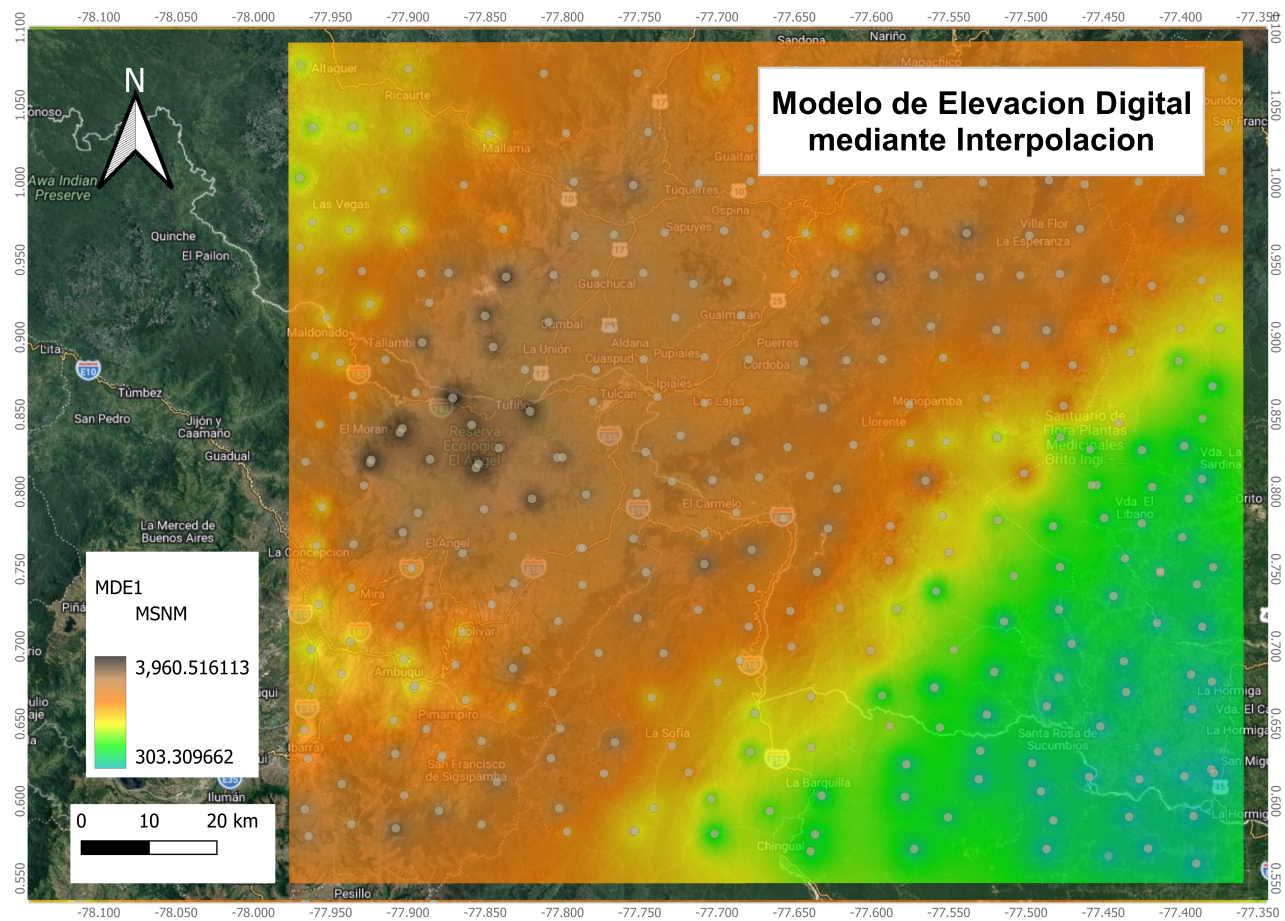


Figure 1: Figura 1. Modelo de Elevacion Digital generado unicamente con el algoritmo IDW, evidenciando la baja correlacion entre los puntos tomados y el modelo generado

2 Algoritmo de interpolacion con correccion SAGA:

En este caso, el uso de un software especializado como SAGA, que implementa algoritmos de interpolacion mas avanzados y con correcciones topograficas, permite generar una representacion mas realista del relieve, con una mejor correlacion entre los puntos y el modelo generado (Figura 2). Esto gracias a que considera de manera uniforme la influencia de los puntos tomados, los espacios vacios entre ellos segun la topografia y la suavizacion de las superficies, concretando una imagen mas fidedigna con correcciones en zonas de cambio brusco en las elevaciones.

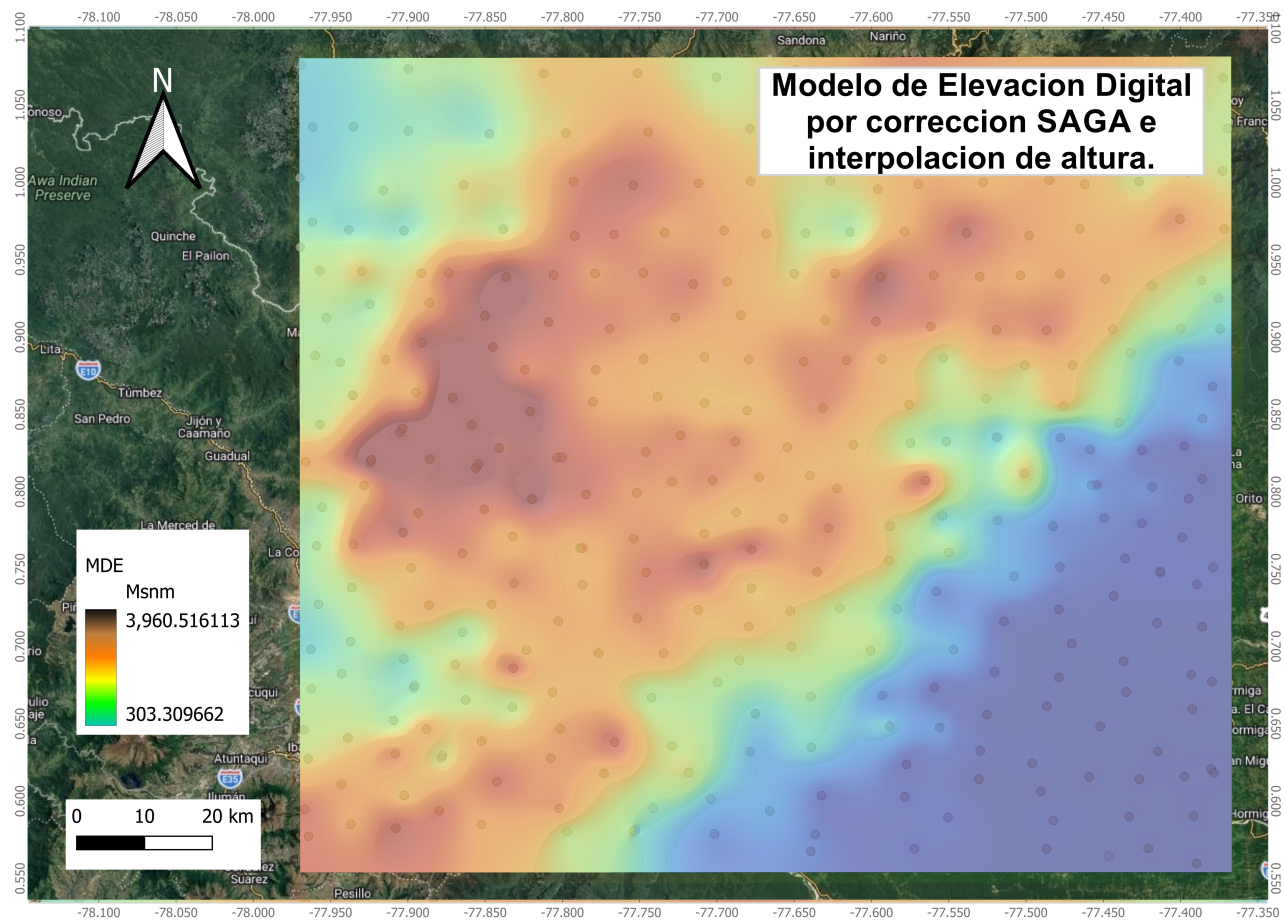


Figure 2: Figura 2. Modelo de Elevacion Digital generado con SAGA, evidenciando una superficie mucho mas suavizada, con continuidad en todo el terreno

```
# 1. IMPORTACIÓN DE LIBRERÍAS
import ee
import folium
from IPython.display import display, IFrame

# 2. AUTENTICACIÓN
# Este comando abre una ventana en el navegador para dar permisos a Google.
# Solo es necesario correrlo la primera vez o si el token de acceso expira.
ee.Authenticate()

# 3. INICIALIZACIÓN DEL PROYECTO
# Reemplaza "ee-giswag" con el ID de tu proyecto en Google Cloud
ee.Initialize(project="gee-wilder-clavijo")

print("Google Earth Engine se ha inicializado con éxito!")
```

Google Earth Engine se ha inicializado con éxito!

```

# -----
# 1. IMPORTACIÓN DE LIBRERÍAS
# -----
import ee
import os
import urllib.request # Librería nativa de Python para descargar URLs
import zipfile         # Librería nativa de Python para descomprimir

# -----
# 2. CARGA DEL MDE Y ÁREA DE DESCARGA
# -----
srtm = ee.Image("USGS/SRTMGL1_003")

# Definimos el punto y el área de interés (Bounding Box de ~30x30 km)
ipiales_punto = ee.Geometry.Point([-77.64, 0.83])
roi_ipiales = ipiales_punto.buffer(15000).bounds()

# -----
# 3. CONFIGURACIÓN DE RUTAS LOCALES
# -----
directorio_salida = "data_heavy/raster"
os.makedirs(directorio_salida, exist_ok=True)

# Archivo temporal donde caerá el ZIP que nos envía Google
ruta_zip = os.path.join(directorio_salida, "mde_ipiales_temp.zip")

# -----
# 4. PETICIÓN NATIVA A EARTH ENGINE (SIN DEPENDENCIAS EXTRA)
# -----
print("Solicitando el enlace de descarga a los servidores de Google...")

# getDownloadURL() es el método nativo de la API de EE.
# Empaquetamos nuestro recorte a 30m en un archivo comprimido.
url_descarga = srtm.getDownloadURL({
    'name': 'mde_ipiales_srtm', # Nombre interno del archivo .tif
    'scale': 30,                # Resolución espacial original (30m)
    'crs': 'EPSG:4326',         # Coordenadas geográficas WGS84
    'region': roi_ipiales       # Filtro espacial obligatorio
})

print(f"Enlace generado. Iniciando descarga...")

# -----
# 5. DESCARGA Y EXTRACCIÓN AUTOMÁTICA
# -----
# Descargamos el archivo ZIP desde la URL generada
urllib.request.urlretrieve(url_descarga, ruta_zip)

print("Descarga finalizada. Descomprimiendo el archivo GeoTIFF...")

```

```

with zipfile.ZipFile(ruta_zip, 'r') as zip_ref:
    # Obtenemos el nombre del archivo que Google metió en el ZIP
    # Probablemente: "mde_ipiales_srtm.elevation.tif"
    nombre_original_ee = zip_ref.namelist()[0]

    # Extraemos el contenido
    zip_ref.extractall(directorio_salida)

# Definimos las rutas para el renombrado
ruta_antigua = os.path.join(directorio_salida, nombre_original_ee)
ruta_nueva = os.path.join(directorio_salida, "mde_ipiales_srtm_elevation.tif")

# Si ya existe el archivo destino, lo borramos para sobrescribir
if os.path.exists(ruta_nueva):
    os.remove(ruta_nueva)

# Ahora sí renombramos el archivo original
if os.path.exists(ruta_antigua):
    os.rename(ruta_antigua, ruta_nueva)

# Limpieza del disco: borramos el ZIP porque ya tenemos el .tif
os.remove(ruta_zip)

print(f"\nPROCESO COMPLETADO CON ÉXITO")
print(f"El MDE de Ipiales ha sido renombrado a: {ruta_nueva}")

```

Solicitando el enlace de descarga a los servidores de Google...
 Enlace generado. Iniciando descarga...
 Descarga finalizada. Descomprimiendo el archivo GeoTIFF...

PROCESO COMPLETADO CON ÉXITO

El MDE de Ipiales ha sido renombrado a: data_heavy/raster/mde_ipiales_srtm_elevation.tif

```

from osgeo import gdal
import rasterio
from rasterio.plot import show
import matplotlib.pyplot as plt

# -----
# 1. CONFIGURACIÓN DE RUTAS Y REPROYECCIÓN (EPSG:9377)
# -----

# Definimos los archivos de entrada y salida
ruta_mde_4326 = "data_heavy/raster/mde_ipiales_srtm_elevation.tif"
ruta_mde_9377 = "data_heavy/raster/mde_ipiales_9377_py.tif"

# gdal.Warp realiza la reproyección al sistema Origen Nacional de Colombia.
# Se captura el objeto 'ds_warp' para asegurar su cierre posterior.
ds_warp = gdal.Warp(ruta_mde_9377, ruta_mde_4326, dstSRS="EPSG:9377")

```

```

# CRÍTICO PARA CUARTO/RETICULATE:
# Se iguala a None para cerrar el puntero de C++, forzar la escritura a disco
# y liberar el archivo para que otros procesos (como DEMProcessing) puedan abrirlo.
ds_warp = None

# -----
# 2. CÁLCULO DE PRODUCTOS TOPOGRÁFICOS (PENDIENTE, ORIENTACIÓN Y SOMBREADO)
# -----

ruta_pendiente = "data_heavy/raster/mde_ipiales_slope_py.tif"
ruta_orientacion = "data_heavy/raster/mde_ipiales_aspect_py.tif"
ruta_sombreado = "data_heavy/raster/mde_ipiales_hillshade_py.tif"

# Pendiente (Slope): Calculada sobre el ráster proyectado en metros.
ds_slope = gdal.DEMProcessing(ruta_pendiente, ruta_mde_9377, "slope")
ds_slope = None # Liberar memoria y cerrar archivo

# Orientación (Aspect): Determina la dirección cardinal de las laderas.
ds_aspect = gdal.DEMProcessing(ruta_orientacion, ruta_mde_9377, "aspect")
ds_aspect = None # Liberar memoria y cerrar archivo

# Sombreado (Hillshade): Modelo de iluminación con Azimut 315° y Altitud 45°.
ds_hill = gdal.DEMProcessing(ruta_sombreado, ruta_mde_9377, "hillshade", azimuth=315, altitude=45)
ds_hill = None # Liberar memoria y cerrar archivo

# -----
# 3. RENDERIZADO VISUAL CARTOGRÁFICO
# -----

# Al usar el gestor de contexto 'with', aseguramos el cierre automático de los archivos.
with rasterio.open(ruta_mde_9377) as mde_src, rasterio.open(ruta_sombreado) as hill_src:
    fig, ax = plt.subplots(figsize=(10, 10))

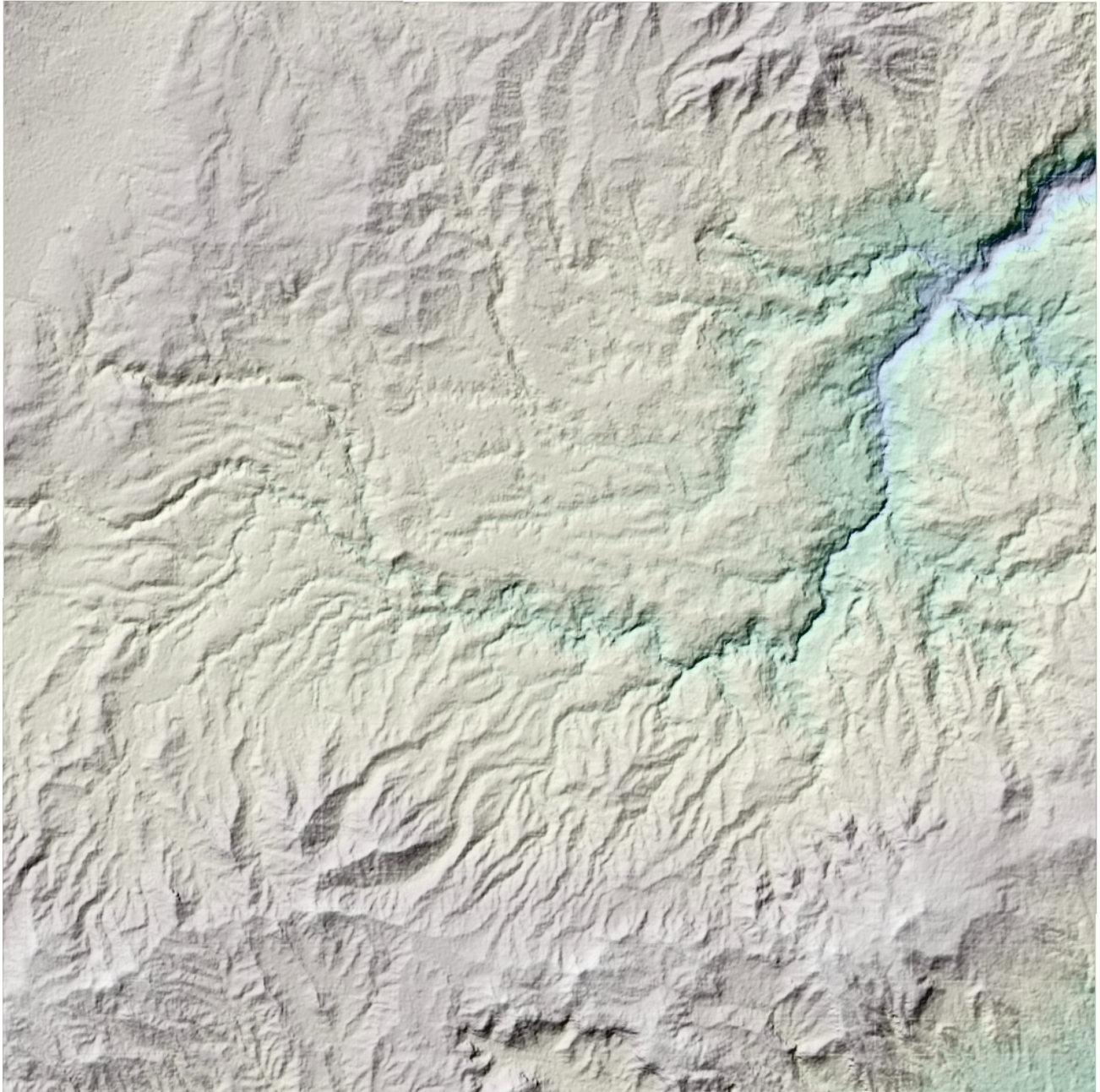
    # 1. Capa base: Modelo de sombras en escala de grises.
    show(hill_src, ax=ax, cmap='Greys_r')

    # 2. Capa superior: Elevación (MDE) con paleta 'terrain' y transparencia del 40%.
    # Esto permite visualizar la topografía sobre el relieve sombreado.
    show(mde_src, ax=ax, cmap='terrain', alpha=0.4)

    # Configuración de estética del mapa
    ax.set_title("Relieve sombreado y elevación - Ipiales (Python/GDAL)")
    ax.set_axis_off()
    plt.show()

```

Relieve sombreado y elevación - Ipiales (Python/GDAL)



Finalmente, en este ejercicio de reproducibilidad demuestra las facilidades que ofrecen las librerías dedicadas a la gestión de datos geospaciales desde su obtención gracias a la API de Earth Engine, hasta su reproyección y cálculo de productos cartográficos con GDAL, permitiendo la generación de mapas con un modelo automatizado y con una representación visual mucho más atractiva.